

Lecture 18: Transport Layer, UDP, and Reliable Data Transfer

From host-to-host to process-to-process — and how to make it reliable

EC 441 — Introduction to Computer Networking
Boston University

Spring 2026

Learning Objectives

By the end of this lecture, you should be able to:

- Explain what the **transport layer** adds over the network layer
- Describe **multiplexing and demultiplexing** using ports and sockets
- Describe the **UDP header** and explain why UDP is the right choice for some applications
- Explain why reliable transfer requires more than just retransmitting lost packets
- Analyze the **link utilization** of Stop-and-Wait and explain why pipelining is necessary
- Trace **Go-Back-N** and **Selective Repeat** through a packet loss event
- Explain the GBN vs. SR design tradeoff and identify which ideas **TCP** inherits from each

Key Acronyms

Acronym	Meaning
ACK	Acknowledgment
ARQ	Automatic Repeat reQuest
BDP	Bandwidth-Delay Product
GBN	Go-Back-N
MSS	Maximum Segment Size

Acronym	Meaning
RDT	Reliable Data Transfer
RTT	Round-Trip Time
SR	Selective Repeat
UDP	User Datagram Protocol
S&W	Stop-and-Wait

From the Network Layer to the Transport Layer

L13–L16: Routing — how packets move from router to router

L17: The IPv4 datagram — what every router actually reads

L18–L20: Transport layer — process-to-process delivery

Network layer (IP):

Delivers a datagram from **host** to **host** —
best-effort, no guarantees

Transport layer (TCP/UDP):

Delivers data from **process** to **process** —
with optional reliability

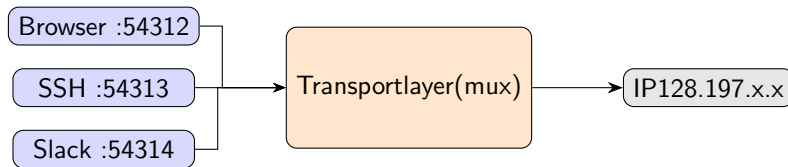
The IPv4 Protocol field (TCP=6, UDP=17) is the hand-off point between the two layers

Physical	Link	Network	Transport	Application
bits	frames	datagrams	segments	messages

5-layer Internet model (TCP/IP). OSI adds Session and Presentation between Transport and Application.

Multiplexing and Demultiplexing

A single host runs many networked processes simultaneously, all sharing one IP address. **Ports** are 16-bit numbers that identify endpoints within a host.



Multiplexing (sender):

Collect data from multiple sockets; add port headers; hand to network layer

Demultiplexing (receiver):

Read destination port; deliver each arriving segment to the correct socket

Port ranges

Range	Name	Assigned by
0–1023	Well-known	IANA
1024–49151	Registered	Convention
49152–65535	Ephemeral	OS (client side)

Port	Proto	Service
22	TCP	SSH
53	UDP/TCP	DNS
80	TCP	HTTP
443	TCP	HTTPS
67/68	UDP	DHCP

Socket = OS abstraction for one communication endpoint

Identified by a **5-tuple**:

- 1 Protocol (TCP or UDP)
- 2 Local IP address
- 3 Local port
- 4 Remote IP address
- 5 Remote port

For UDP: demux key = (dst IP, dst port)

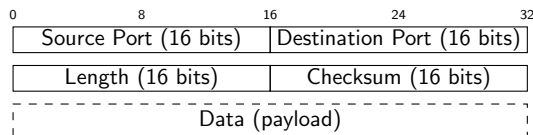
For TCP: demux key = full 5-tuple

⇒ thousands of TCP connections
can share port 80

UDP: The Minimal Transport Protocol

*IP is best-effort at the network layer. UDP extends that philosophy up one layer: reliability, ordering, and flow control are the **application's problem**, not the protocol's.*

UDP adds exactly **8 bytes** of header — port numbers plus a checksum.



No connection setup
No delivery guarantee
No ordering guarantee
No congestion control

Yes port-based demultiplexing
Yes optional checksum
Yes immediate send (no handshake)
Yes supports multicast/broadcast

The UDP Checksum

- Ones'-complement sum over a **pseudo-header** (source IP, destination IP, protocol = 17, UDP length) plus the UDP header and data
- **Why include IP addresses?** Guards against misdelivered datagrams — a packet routed to the wrong host will fail the checksum even if the UDP header is intact
- Same principle as L17: ones'-complement sum gives $d_{\min} = 2$, detecting all single-bit errors
- **Optional in IPv4** (value 0x0000 = “not computed”); **mandatory in IPv6** (IPv6 dropped the IP-layer header checksum entirely)

UDP checksums are lightweight but not strong — no protection against reordering or multi-bit errors. Higher-layer protocols (TLS, application) add stronger integrity when needed.

Why Use UDP?

Low-latency interactive

VoIP, video calls, online games
A dropped frame $<$ a delayed frame.
Retransmitting stale audio is useless.

Query-response

DNS, NTP, DHCP
One datagram out, one datagram back.
TCP setup adds ≥ 1.5 RTT for no benefit.

Multicast / broadcast

UDP can address groups.
TCP requires point-to-point connections.

Application-controlled reliability

QUIC (HTTP/3), WebRTC
App implements its own loss recovery, congestion control, and TLS on top of UDP, bypassing TCP's head-of-line blocking.

Tunneling and VPNs

WireGuard, DTLS
Avoids “TCP-over-TCP” pathology: outer TCP retransmits interact badly with inner TCP congestion control.

UDP is not “TCP with reliability removed” — it is a different design point with its own strengths.

Demo: Active UDP Sockets and DNS

```
# See active UDP sockets and owning processes
ss -u -p

# A standard DNS query (UDP port 53)
dig +short bu.edu

# DNS over TCP explicitly
dig +tcp bu.edu

# Large TXT record -- may trigger automatic TCP fallback
dig TXT google.com
```

Discussion: Why does DNS fall back to TCP for large responses?

Classic DNS truncates UDP responses at 512 bytes. EDNS0 extended this, but responses larger than the path MTU still trigger TCP. DNSSEC signatures can be large enough to require it.

[Wikipedia: User Datagram Protocol] [Wikipedia: Domain Name System]

The Reliable Data Transfer Problem

The network layer delivers packets **best-effort** — bits can be corrupted, packets can be lost. Many applications (file transfer, web, SSH) cannot tolerate this.

Naïve answer: “just retransmit on failure.” But this raises harder questions:

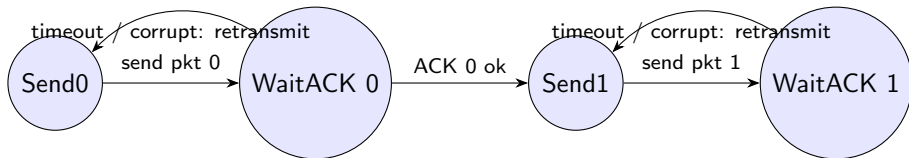
- How does the sender know something went wrong?
- How does it know *which* packet to retransmit?
- How does it avoid retransmitting packets that already arrived?
- How does it keep the link busy while waiting for acknowledgments?

The building blocks:

- **Sequence numbers:** label packets; detect gaps and duplicates
- **Acknowledgments (ACKs):** receiver confirms correct receipt
- **Timers:** if no ACK within timeout, assume loss and retransmit
- **Checksums:** detect corrupted packets

Stop-and-Wait: The Simplest Protocol

Send one packet. Wait for ACK. Then send the next.



Only **1-bit sequence numbers** needed (0 and 1 alternating)

⇒ called the **Alternating Bit Protocol**

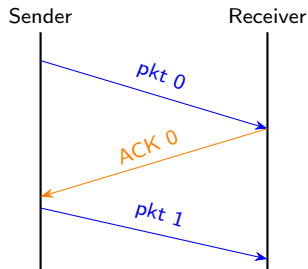
Receiver: if seq num = expected ⇒ deliver and ACK

If seq num ≠ expected ⇒ duplicate, re-send last ACK

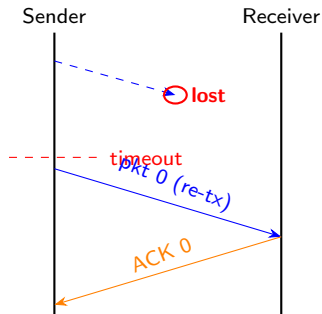
Correct — but how fast?

Stop-and-Wait: What If the Data Packet Is Lost?

Normal case (no loss):



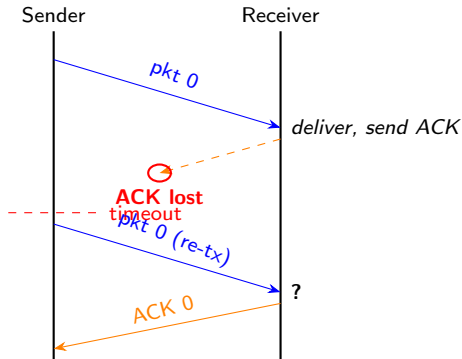
Data packet lost:



Data loss is handled cleanly: sender times out, retransmits, receiver ACKs. No ambiguity.

Stop-and-Wait: What If the ACK Is Lost?

ACK lost:



The receiver now sees **pkt 0** again.

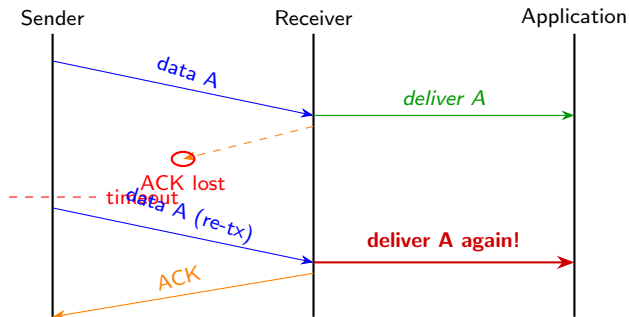
What should it do?

- It already delivered pkt 0 to the application
- If it delivers again $\Rightarrow$ **duplicate data**
- If it discards $\Rightarrow$ correct, but *how does it know to discard?*

The receiver needs a way to distinguish:
“new data” vs. **“retransmit I already have”**
 $\Rightarrow$ this is exactly what sequence numbers solve.

Why Sequence Numbers Are Necessary

Without sequence numbers: receiver cannot tell retransmit from new data.



With 1-bit sequence numbers: receiver is waiting for seq 1; sees seq 0 again $\Rightarrow$ duplicate $\Rightarrow$ discard, re-send ACK 0. Application receives data A exactly once. ✓

This is why even Stop-and-Wait needs sequence numbers, even though it only ever has one packet outstanding.

Stop-and-Wait: Link Utilization

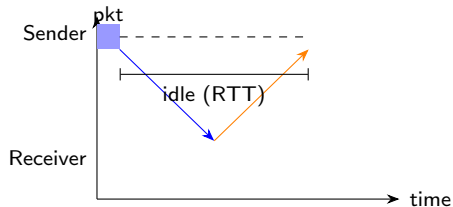
Example: satellite link

- Bandwidth: 1 Gb/s
- RTT: 600 ms (geostationary)
- Packet size: 1,500 bytes = 12,000 bits

Transmit time: $\frac{12,000}{10^9} = 12 \mu\text{s}$

$$U = \frac{t_{\text{transmit}}}{RTT + t_{\text{transmit}}} = \frac{0.012 \text{ ms}}{600 \text{ ms}} \approx \mathbf{0.002\%}$$

The sender spends 99.998% of its time waiting.
A 1 Gb/s link delivers $\approx \mathbf{20 \text{ kb/s}}$.



The packet is tiny relative to the round-trip time.

Bandwidth-Delay Product and Pipelining

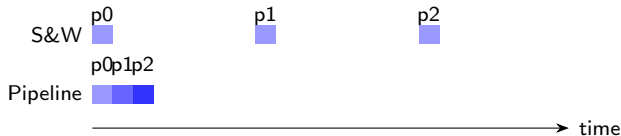
Bandwidth-delay product (BDP): the number of bits that could be in flight on a fully-utilized link.

$$\text{BDP} = \text{Bandwidth} \times \text{RTT} = 1 \text{ Gb/s} \times 600 \text{ ms} = 600 \text{ Mb}$$

Stop-and-Wait keeps only **one packet** in flight ($\approx 12,000$ bits). To fill the pipe, we need:

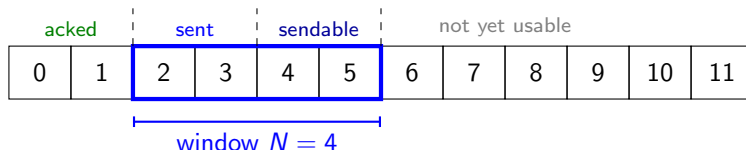
$$\text{Window size} \geq \frac{\text{BDP}}{\text{Packet size}} = \frac{600 \text{ Mb}}{12,000 \text{ bits}} \approx \mathbf{50,000} \text{ packets}$$

Pipelining: send up to N packets before waiting for ACK — the window keeps the link busy.



Two pipelining protocols: **Go-Back-N** and **Selective Repeat** — they differ in what happens when a packet is lost.

Go-Back-N: Sender Window

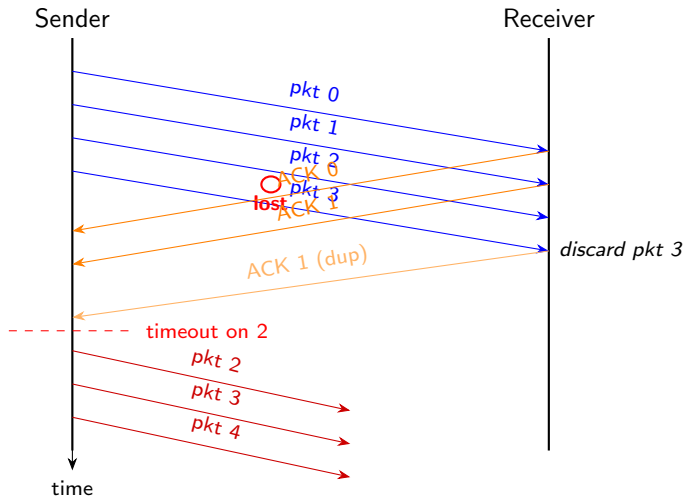


Key rules:

- ACKs are **cumulative**: ACK n = "received all packets through n "
- **One timer** for the oldest unacknowledged packet
- On timeout: **retransmit all packets in the window**
- Receiver: out-of-order packets are **discarded** — no buffering needed
- Sequence numbers: window size $N \leq 2^k - 1$ for k -bit sequence numbers

Go-Back-N: Packet Loss Trace

Window size $N = 4$. Packet 2 is lost.



One lost packet $\Rightarrow$ retransmit all packets in the window. Simple receiver; wasteful under loss.

Selective Repeat: Protocol

Fix GBN's inefficiency: the receiver **buffers out-of-order packets**; the sender retransmits **only the lost packet**.

Sender:

- Same window concept as GBN
- **Individual timer per packet**
- On timeout for packet n : retransmit *only* packet n
- ACKs are **individual** (not cumulative)

Receiver:

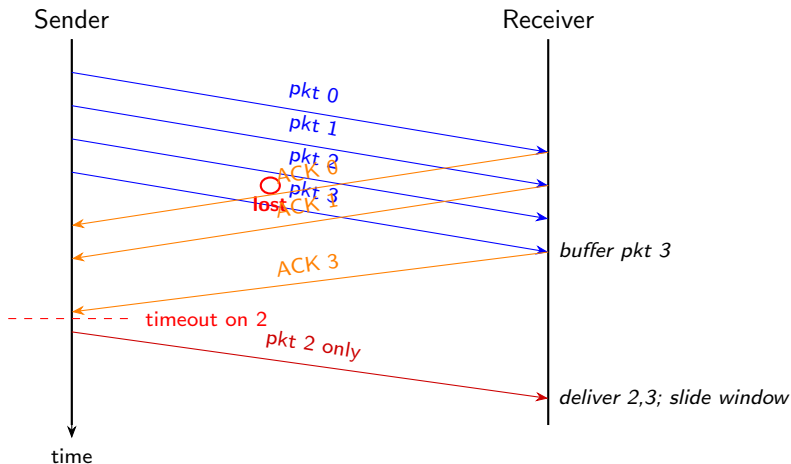
- If packet n is in $[\text{rcv_base}, \text{rcv_base} + N - 1]$:
buffer it, send ACK n
- If packet $n = \text{rcv_base}$:
deliver n + any consecutive buffered packets, slide window
- If duplicate of already-acked packet:
send ACK n again (sender may not have received it)

Sequence number constraint: $N \leq 2^{k-1}$ (half the sequence number space)

Tighter than GBN — needed to prevent the receiver from confusing a new packet with a retransmit of one it already delivered.

Selective Repeat: Same Scenario

Window size $N = 4$. Packet 2 is lost.



One lost packet $\Rightarrow$ one retransmission. More complex receiver; much more efficient under loss.

GBN vs. SR: Comparison

	Go-Back-N	Selective Repeat
Receiver buffer	None needed	Must buffer out-of-order pkts
Retransmit on loss	All packets in window	Only the lost packet
ACK type	Cumulative	Individual
Timers	One (oldest unacked)	One per unacked packet
Sequence number limit	$N \leq 2^k - 1$	$N \leq 2^{k-1}$
Good when	Loss rate is low	Loss rate is significant

TCP borrows from both:

- Cumulative ACKs (GBN-style) — but also supports **selective ACK (SACK)** option
- Retransmits only missing segments when SACK is in use (SR-style)
- Receiver buffer required (like SR) — enables out-of-order delivery within the window

Bridge to TCP: Design Question

We now have the pieces. Consider:

*You want to build a reliable, full-duplex, byte-stream protocol on top of IP.
You have sequence numbers, ACKs, timers, and pipelining.
What design choices would you make?*

Questions you can already answer:

- Cumulative or individual ACKs?
- How to detect loss?
- How large should the window be?

Questions we haven't addressed yet:

- How do you set up and tear down the connection?
- What if the receiver's buffer fills up?
- What if the *network* is overloaded, not just the receiver?

Next lecture: TCP answers all of these.